

Orion Object Storage — Administrator Guide

Version 4.2 | Last updated: March 2026 | Document ID: ORN-ADMIN-042

1. Overview

Orion Object Storage (OOS) is a distributed, S3-compatible object storage system designed for high-throughput analytics workloads. This guide covers installation, configuration, security, monitoring, and troubleshooting for administrators responsible for operating an Orion cluster in production. Orion clusters scale horizontally across storage nodes and use erasure coding to protect data against node failure without the 3x overhead of full replication.

- Erasure-coded storage (default 8+4 scheme) with configurable durability targets
- S3-compatible REST API supporting multipart upload, versioning, and lifecycle rules
- Built-in metrics endpoint compatible with Prometheus scraping
- Pluggable identity providers: LDAP, OIDC, and static API keys
- Cross-region replication with configurable RPO (recovery point objective)

2. Architecture

An Orion cluster consists of three node roles: **gateway nodes** that terminate client connections and route requests, **storage nodes** that hold erasure-coded data shards, and **metadata nodes** that run a Raft-replicated key-value store tracking object locations. A minimum production cluster requires 3 metadata nodes, 2 gateway nodes, and 6 storage nodes.

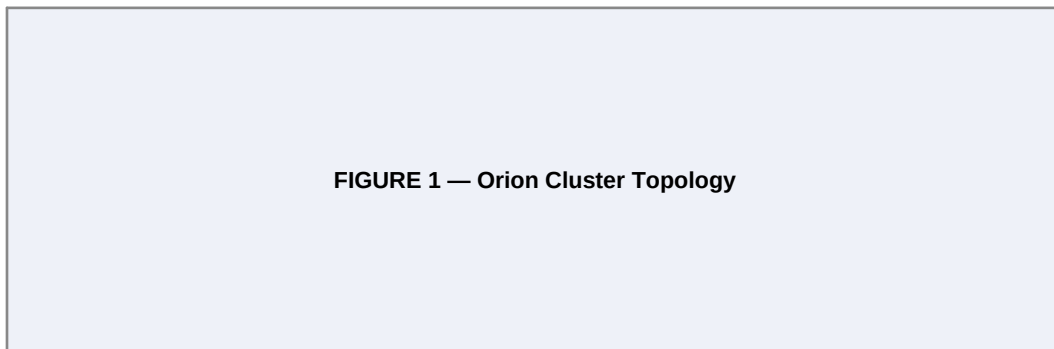


FIGURE 1 — Orion Cluster Topology

Figure 1. Gateway nodes route client requests to storage nodes; metadata nodes coordinate placement via Raft consensus. Arrows indicate the write path for a PUT request.

Clients never talk to storage nodes directly. Every read or write passes through a gateway node, which consults the metadata layer to resolve object placement before forwarding shard requests to the relevant storage nodes in parallel. This design lets Orion add storage capacity without any client-visible reconfiguration.

3. Installation

Orion ships as a single static binary plus a systemd unit file. The steps below assume a Debian-based host with at least 4 vCPUs and 16 GB RAM per storage node.

- 1 Download the release archive for your architecture from the internal package mirror.
- 2 Extract the archive to `/opt/orion` and verify the SHA-256 checksum.
- 3 Copy the sample configuration file to `/etc/orion/orion.yaml`.
- 4 Edit the configuration to set the node role, cluster seed addresses, and storage device paths.
- 5 Enable and start the systemd service on each node in order: metadata nodes first, then storage nodes, then gateway nodes.
- 6 Run `orion-ctl cluster verify` to confirm all nodes have joined successfully.

3.1 Example Installation Commands

```
curl -O https://mirror.internal/orion/orion-4.2.0-linux-amd64.tar.gz
sha256sum -c orion-4.2.0-linux-amd64.tar.gz.sha256
tar -xzf orion-4.2.0-linux-amd64.tar.gz -C /opt/orion
cp /opt/orion/config/orion.sample.yaml /etc/orion/orion.yaml
systemctl enable --now orion-node
orion-ctl cluster verify
```

1. The package mirror URL differs by environment; see the internal deployment wiki for staging and production mirror addresses.

4. Configuration Reference

The table below documents the most commonly modified fields in `orion.yaml`. Fields not listed here retain safe defaults and rarely need to change.

Field	Type	Default	Description
<code>node.role</code>	string	—	One of: gateway, storage, metadata
<code>cluster.seed_addr</code> s	list	—	Comma-separated host:port list used for initial cluster join
<code>storage.erasure_scheme</code>	string	8+4	Data shards + parity shards, e.g. 8+4 or 4+2
<code>storage.device_paths</code>	list	—	Mount points reserved exclusively for Orion data
<code>api.max_object_size_mb</code>	int	5120	Maximum single-object size before multipart is required
<code>auth.provider</code>	string	static	One of: static, ldap, oidc
<code>replication.mode</code>	string	off	One of: off, async, sync
<code>replication.target_rpo_seconds</code>	int	300	Maximum acceptable replication lag when mode is async
<code>metrics.enabled</code>	bool	true	Exposes a Prometheus-compatible <code>/metrics</code> endpoint

Field	Type	Default	Description
metrics.port	int	9090	Port for the metrics HTTP listener

5. API Reference

Orion exposes an S3-compatible REST API on port 8443 (TLS) by default. Full endpoint documentation is published at <https://docs.internal/orion/api>. The most frequently used operations are summarized below.

5.1 Create a Bucket

```
PUT /my-bucket HTTP/1.1
Host: orion.internal:8443
Authorization: AWS4-HMAC-SHA256 Credential=...
x-orion-erasure-scheme: 8+4
```

5.2 Multipart Upload

Objects larger than the configured `api.max_object_size_mb` threshold must be uploaded using the three-step multipart flow: initiate, upload parts, then complete. Each part except the last must be at least 5 MB.

- 1 POST `/my-bucket/my-object?uploads` — returns an `UploadId`
- 2 PUT `/my-bucket/my-object?partNumber=N&uploadId=ID` — upload each part, collecting its ETag
- 3 POST `/my-bucket/my-object?uploadId=ID` — complete the upload with the list of part ETags

5.3 Lifecycle Rules

Lifecycle rules automatically transition or expire objects based on age. Rules are evaluated once per hour by a background sweeper process running on the metadata nodes. See <https://docs.internal/orion/lifecycle> for the full rule schema.

6. Security & Access Control

Orion supports three authentication providers. Static API keys are recommended only for development clusters; production clusters should use LDAP or OIDC so that access can be centrally revoked.

Role	Permissions	Typical Assignment
orion-admin	Full cluster administration, including node join/remove	Platform team leads
orion-operator	Bucket create/delete, lifecycle rule management	Platform on-call engineers
orion-writer	PUT and DELETE objects within assigned buckets	Application service accounts
orion-reader	GET and LIST objects within assigned buckets	Analytics and reporting jobs

7. Monitoring & Alerts

The metrics endpoint exposes counters and gauges compatible with Prometheus. The table below lists the metrics most relevant to on-call troubleshooting, along with recommended alert thresholds.

Metric	Type	Recommended Alert Threshold
orion_shard_repair_queue_depth	gauge	Alert if > 500 for more than 15 minutes
orion_gateway_request_latency_p99_ms	histogram	Alert if > 800ms for more than 5 minutes
orion_storage_node_disk_used_percent	gauge	Alert if > 85%
orion_metadata_raft_leader_changes_total	counter	Alert on any change outside maintenance windows
orion_replication_lag_seconds	gauge	Alert if greater than replication.target_rpo_seconds



Figure 2. The default Orion Grafana dashboard, showing shard repair queue depth and gateway p99 latency over a 24-hour window.

8. Troubleshooting

Symptom	Likely Cause	Recommended Action
503 errors from gateway nodes	Metadata Raft leader election in progress	Wait up to 30s; check orion_metadata_raft_leader_changes_total
Slow uploads for large objects	Multipart part size too small	Increase part size to at least 64 MB
Shard repair queue growing steadily	Storage node failed or disk degraded	Check disk health with orion-ctl storage diag
Replication lag exceeding RPO	Network bandwidth saturation between regions	Check cross-region link utilization

9. Deployment Topologies

Orion supports three reference topologies depending on availability requirements. Choosing the right topology up front avoids a disruptive migration later, since changing the erasure scheme or replication mode requires re-encoding all existing objects.

9.1 Single-Region, Single-Zone

Suitable for development and staging clusters only. All nodes run in one availability zone with erasure scheme 4+2. This topology tolerates up to two simultaneous storage node failures but does not protect against a zone-wide outage.

9.2 Single-Region, Multi-Zone

The recommended production baseline. Storage nodes are spread across at least three availability zones with erasure scheme 8+4, so the cluster tolerates a full zone outage without data loss. Gateway nodes should also be spread across zones behind a regional load balancer.

9.3 Multi-Region with Async Replication

For workloads with regional disaster recovery requirements, a secondary cluster in a second region receives asynchronous replication traffic. The `replication.target_rpo_seconds` field controls the maximum acceptable data loss window if the primary region is lost. Values below 60 seconds are not recommended, since sustained replication lag below that threshold requires dedicated cross-region bandwidth that most deployments do not provision.

10. Backup and Disaster Recovery

Erasur coding protects against storage node failure, but it does not protect against accidental deletion, bucket-level misconfiguration, or logical corruption introduced by a faulty application. Administrators should combine erasure coding with the following practices.

- Enable bucket versioning on any bucket holding data that cannot be regenerated from source.
- Configure lifecycle rules to move noncurrent versions to a lower-cost storage class after 30 days rather than deleting them immediately.
- Schedule a weekly `orion-ctl backup snapshot` job for the metadata store, which is small enough to restore quickly even though object data itself is not duplicated by this snapshot.
- Test restore procedures quarterly. A backup that has never been restored should be treated as untested.

10.1 Restore Procedure

- 1 Provision a clean metadata node set and restore the most recent metadata snapshot.
- 2 Rejoin existing storage nodes to the restored metadata cluster.
- 3 Run `orion-ctl cluster verify` to confirm shard placement matches the restored metadata.
- 4 Run `orion-ctl storage scrub` on all storage nodes to detect and repair any shard inconsistencies.

11. Appendix A — CLI Command Reference

Command	Description
orion-ctl cluster verify	Confirms all configured nodes have joined and are healthy
orion-ctl cluster status	Prints current node roles, Raft leader, and shard repair queue depth
orion-ctl storage diag	Runs disk health diagnostics on a specific storage node
orion-ctl storage scrub	Verifies shard checksums across all storage nodes and repairs mismatches
orion-ctl backup snapshot	Takes a point-in-time snapshot of the metadata store
orion-ctl bucket lifecycle list	Lists active lifecycle rules for a bucket

12. References

- Orion API Reference — <https://docs.internal/orion/api>
- Orion Lifecycle Rule Schema — <https://docs.internal/orion/lifecycle>
- Erasure Coding Design Document (ORN-DESIGN-011)
- Prometheus Alerting Best Practices — <https://prometheus.io/docs/alerting/latest/overview/>